

Because a call to `map2` always evaluates its second argument, our `many` function will never terminate! That's no good. This indicates that we need to make `product` and `map2` non-strict in their second argument:

```
def product[A,B] (p: Parser[A], p2: => Parser[B]): Parser[(A,B)]

def map2[A,B,C] (p: Parser[A], p2: => Parser[B]) (
    f: (A,B) => C): Parser[C] =
    product(p, p2) map (f.tupled)
```



EXERCISE 9.5

We could also deal with non-strictness with a separate combinator like we did in chapter 7. Try this here and make the necessary changes to your existing combinators. What do you think of that approach in this instance?

Now our implementation of `many` should work fine. Conceptually, `product` should have been non-strict in its second argument anyway, since if the first `Parser` fails, the second won't even be consulted.

We now have good combinators for parsing one thing followed by another, or multiple things of the same kind in succession. But since we're considering whether combinators should be non-strict, let's revisit the `or` combinator from earlier:

```
def or[A] (p1: Parser[A], p2: Parser[A]): Parser[A]
```

We'll assume that `or` is left-biased, meaning it tries `p1` on the input, and then tries `p2` only if `p1` fails.⁹ In this case, we ought to make it non-strict in its second argument, which may never even be consulted:

```
def or[A] (p1: Parser[A], p2: => Parser[A]): Parser[A]
```

9.3 Handling context sensitivity

Let's take a step back and look at the primitives we have so far:

- `string(s)`—Recognizes and returns a single `String`
- `slice(p)`—Returns the portion of input inspected by `p` if successful
- `succeed(a)`—Always succeeds with the value `a`
- `map(p) (f)`—Applies the function `f` to the result of `p`, if successful
- `product(p1, p2)`—Sequences two parsers, running `p1` and then `p2`, and returns the pair of their results if both succeed
- `or(p1, p2)`—Chooses between two parsers, first attempting `p1`, and then `p2` if `p1` fails

⁹ This is a design choice. You may wish to think about the consequences of having a version of `or` that always runs both `p1` and `p2`.

Using these primitives, we can express repetition and nonempty repetition (`many`, `listOfN`, and `many1`) as well as combinators like `char` and `map2`. Would it surprise you if these primitives were sufficient for parsing *any* context-free grammar, including JSON? Well, they are! We'll get to writing that JSON parser soon, but what *can't* we express yet?

Suppose we want to parse a single digit, like `'4'`, followed by *that many* `'a'` characters (this sort of problem should feel familiar from previous chapters). Examples of valid input are `"0"`, `"1a"`, `"2aa"`, `"4aaaa"`, and so on. This is an example of a context-sensitive grammar. It can't be expressed with `product` because our choice of the second parser *depends on* the result of the first (the second parser depends on its context). We want to run the first parser, and then do a `listOfN` using the number extracted from the first parser's result. Can you see why `product` can't express this?

This progression might feel familiar to you. In past chapters, we encountered similar expressiveness limitations and dealt with it by introducing a new primitive, `flatMap`. Let's introduce that here (and we'll add an alias to `ParserOps` so we can write parsers using for-comprehensions):

```
def flatMap[A,B](p: Parser[A])(f: A => Parser[B]): Parser[B]
```

Can you see how this signature implies an ability to sequence parsers where each parser in the chain depends on the output of the previous one?



EXERCISE 9.6

Using `flatMap` and any other combinators, write the context-sensitive parser we couldn't express earlier. To parse the digits, you can make use of a new primitive, `regex`, which promotes a regular expression to a `Parser`.¹⁰ In Scala, a string `s` can be promoted to a `Regex` object (which has methods for matching) using `s.r`, for instance, `"[a-zA-Z_][a-zA-Z0-9_]*".r`.

```
implicit def regex(r: Regex): Parser[String]
```



EXERCISE 9.7

Implement `product` and `map2` in terms of `flatMap`.



EXERCISE 9.8

`map` is no longer primitive. Express it in terms of `flatMap` and/or other combinators.

¹⁰ In theory this isn't necessary; we could write out `"0" | "1" | ... "9"` to recognize a single digit, but this isn't likely to be very efficient.